Vietnam National University

Ho Chi Minh City University of Technology

Faculty of Computer Science and Engineering



ASSIGNMENT 1

Deep Learning and Its Applications (CO3133)

Lecturer: LÊ THÀNH SÁCH

Student Name	Student ID
Đỗ Đăng Khoa	2352554
Trần Hoàng Vỹ Khang	2352502
Dương Đăng Khoa	2352551

Ho Chi Minh City, October 2026

Table of Contents

1 Problem and Data Description **3**
1.1 Problem Formulation . . . 3
1.2 Dataset Description and Dataset Roles . . . 4
1.3 Exploratory Data Analysis (EDA) . . . 4
1.4 Data Splitting . . . 6
1.5 Preprocessing and Input Representations . . . 6

2 Methodology **7**
2.1 Pipeline Overview . . . 7
2.2 Baseline and Main Methods . . . 8
2.2.1 Linear / Softmax Classifier (Baseline) . . . 8
2.2.2 Multilayer Perceptron (MLP) . . . 9
2.2.3 Convolutional Neural Network (CNN) . . . 9
2.2.4 Recurrent Neural Networks (LSTM and GRU) . . . 10
2.2.5 Transformer Encoder Classifier . . . 11
2.3 Training Setup . . . 11
2.4 Experimental Design . . . 12

3 Implementation Results **13**
3.1 Training Behavior . . . 13
3.2 Quantitative Results . . . 16
3.3 Qualitative Results and Error Analysis . . . 16
3.3.1 Detailed Analysis of Error Groups . . . 19
3.4 Comparison and Discussion . . . 20
3.5 Optional Extension (Section 11.2): LSTM vs. GRU Comparison . . . 21
3.6 Limitations, Generalization, and Future Work . . . 22
3.7 Conclusion (Answer to the Research Question) . . . 22

4 Conclusion **23**

5 AI Usage Disclosure **23**
5.1 Group Member Roles and AI Usage Overview . . . 24
5.2 Detailed Disclosure Entries . . . 24
5.2.1 Entry 1: Review and Cleanup of the M1 Draft Training Pipeline . 24
5.2.2 Entry 2: Review of the Prototype MLP–CNN Branch . . . 25
5.2.3 Entry 3: Review of M2 Models, Sequence Helper, and Timing Isolation 26
5.2.4 Entry 4: Combined Figures and Scientific Audit of the Report . . 26

List of Figures

1 Three representative training images per class (one class per column) across the 10 Fashion-MNIST categories. 5
2 Per-class mean images across the 10 Fashion-MNIST categories. T-shirt/top, Pullover, Coat, and Shirt average to nearly the same torso silhouette, predicting the dominant confusion cluster. 5
3 End-to-end experimental pipeline from raw Fashion-MNIST images . . . 7
4 Training and validation Cross-Entropy Loss (left column) and Macro-F1 (right column) over 15 epochs for all six architectures on Fashion-MNIST (seed 42). Vertical dotted lines mark the best validation Macro-F1 checkpoint selected for test evaluation. 14
5 CNN misclassifications on the test split (true and predicted category with confidence), showing errors concentrated among visually overlapping upper-body garments. 17
6 Correct CNN test examples, including clean-silhouette items (Trouser, Bag, Sandal) and challenging long-sleeved upper-body garments (Shirt, Pullover, Coat). 17
7 Row-normalized confusion matrices on the 10,000-image Fashion-MNIST test split across Linear, MLP, CNN, LSTM, GRU, and Transformer. . . . 18

List of Tables

1 Dataset roles in this study. 4
2 Overall quantitative comparison on the Fashion-MNIST held-out test set ($N = 10{,}000$, seed 42). 16
3 Per-class test F1 scores on Fashion-MNIST (seed 42), ordered by Linear baseline difficulty. 16
4 Error-group frequencies across all six models on the official test split ($N = 10{,}000$; percentages are shares of each model's total errors). 19
5 Controlled recurrent comparison: LSTM versus GRU on Fashion-MNIST (seed 42). 21
6 Group contributions, reported AI use, and verification responsibilities. . . 24

1 Problem and Data Description

1.1 Problem Formulation

- **Real-world motivation:** Automated garment classification underpins e-commerce catalog indexing, visual search, and retail inventory tagging, where product images frequently arrive as studio thumbnails without reliable category metadata. Methodologically, classifying low-resolution apparel imagery provides a controlled benchmark to investigate how **input data representation** (one-dimensional flattened pixel vectors vs. two-dimensional spatial grids vs. one-dimensional horizontal row sequences) and **architectural inductive bias** govern classification accuracy, parameter efficiency, and inference latency when all optimization settings are held constant.

- **Input:** A single-channel grayscale image $X \in \mathbb{R}^{1\times 28\times 28}$ (784 pixel intensities when flattened) depicting a centered article of clothing or footwear.

- **Output:** A categorical class index $\hat{y} \in \{0, 1, \ldots, 9\}$ corresponding to one of 10 apparel categories (`T-shirt/top`, `Trouser`, `Pullover`, `Dress`, `Coat`, `Sandal`, `Shirt`, `Sneaker`, `Bag`, or `Ankle boot`), obtained from unnormalized class logits $z = f_\theta(X) \in \mathbb{R}^{10}$.

- **Prediction unit:** One independent categorical prediction per 28×28 image.

- **Mathematical formulation:** Given a labeled training set $\mathcal{D}_{\text{train}} = \{(X_i, y_i)\}_{i=1}^{N_{\text{train}}}$, a parametric neural network $f_\theta : \mathbb{R}^{1\times 28\times 28} \to \mathbb{R}^{10}$ maps each image X_i to unnormalized logits $z_i = f_\theta(X_i)$. The model parameters θ are optimized by minimizing the multi-class Cross-Entropy loss over the training split:

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \log \frac{\exp(z_{i,y_i})}{\sum_{c=0}^{9} \exp(z_{i,c})}.$$

  At inference time, the predicted category is assigned via the maximum-logit rule $\hat{y}_i = \arg\max_{c\in\{0,\ldots,9\}} z_{i,c}$.

- **Main difficulties:**

  1. **Silhouette overlap among upper-body garments:** Four categories—`T-shirt/top` (0), `Pullover` (2), `Coat` (4), and `Shirt` (6)—share nearly identical outer torso boundaries and differ primarily in fine interior cues such as collars, button plackets, sleeve seams, and knit textures.
  2. **Low spatial resolution and absence of color:** Downsampling studio photographs to 28×28 single-channel grayscale eliminates chromatic cues and compresses fine structural details into 1–2 pixel intensity transitions.
  3. **Intra-class geometric variation:** Categories such as `Bag` (8), `Sandal` (5), and `Ankle boot` (9) vary substantially in strap geometry, handle orientation, and heel profile.

1.2 Dataset Description and Dataset Roles

In accordance with Section 10 of the Course Handbook, the role of each candidate dataset in this study is defined below:

Table 1: Dataset roles in this study.

Dataset	Role in This Study	Scope and Usage
MNIST	**Development and Verification Only**	Used strictly during initial pipeline setup to verify tensor shapes and training loop integrity. No experimental claims are reported on MNIST.
Fashion-MNIST	**Primary Benchmark Dataset**	Used for all main experimental comparisons, exploratory data analysis, training dynamics, confusion matrices, and qualitative error analyses.
CIFAR-10	**Not Evaluated (Optional)**	Omitted in favor of a controlled LSTM vs. GRU recurrent architecture comparison on Fashion-MNIST under Section 11.2.

- **Source, version, and license:** Fashion-MNIST was introduced by Xiao, Rasul, and Vollgraf (Zalando Research, 2017) under the MIT License.
- **Collection and labeling process:** Derived from product photographs on Zalando's e-commerce platform. Each original color image was converted to grayscale, cropped to its bounding box, resized into a 28×28 frame while preserving aspect ratio, and assigned its catalog product category label.
- **Sample counts and annotations:** 70,000 images in total—60,000 official development images and 10,000 official held-out test images—distributed across 10 mutually exclusive classes.
- **Storage size and format:** Compressed IDX binary archives occupying approximately 30 MB on disk.
- **Bias and limitations:** Studio-lit, clean-background thumbnails (with 50.2% exact-zero black background pixels) do not reflect real-world background clutter, illumination shifts, partial occlusions, or articulated human pose deformations.

1.3 Exploratory Data Analysis (EDA)

Our exploratory analysis of Fashion-MNIST examines input statistics, class balance, and spatial structure prior to model training:

1. **Input dimensions and pixel statistics:** Every sample is a $1 \times 28 \times 28$ grayscale raster (784 features when flattened) with raw 8-bit integer intensities in $[0, 255]$. When linearly rescaled to $[0.0, 1.0]$, the global pixel mean and standard deviation across the 60,000 official development images are $\mu = 0.2860$ and $\sigma = 0.3530$ (recomputing these statistics solely over our 54,000-image stratified training subset yields the exact

same $\mu = 0.2860$ and $\sigma = 0.3530$ to four decimal places). Exactly 50.2% of all pixels are 0.0 (pure black background), yielding a strongly bimodal intensity distribution.

2. **Class distribution and imbalance analysis:** Every class $c \in \{0, \ldots, 9\}$ contains exactly 6,000 official development images (5,400 training and 600 validation images after our stratified 90/10 split) and exactly 1,000 official test images (10.0% per class across all splits). This yields an imbalance ratio of 1.0000 ($\max_c N_c / \min_c N_c = 1.0$) and a normalized Shannon entropy of 1.0000. Because the dataset is perfectly balanced, class reweighting or resampling is unnecessary, a uniform random classifier achieves 10.0% accuracy, and overall Accuracy and Macro-F1 remain closely aligned.

3. **Representative samples and per-class mean templates:**

   - **Figure 1** displays a 3×10 grid of representative training samples (three randomly sampled training images per class, one class per column), illustrating intra-class variability in sleeve cut, surface pattern, footwear cutout, and bag-handle geometry.
   - **Figure 2** displays the per-class mean images across the 10 categories. Visually, `Trouser`, `Bag`, `Sandal`, `Sneaker`, and `Ankle boot` each average to a sharp, distinctive silhouette, whereas `T-shirt/top`, `Pullover`, `Coat`, and `Shirt` average to nearly the same diffuse torso template—anticipating where classification errors will concentrate across all model families.

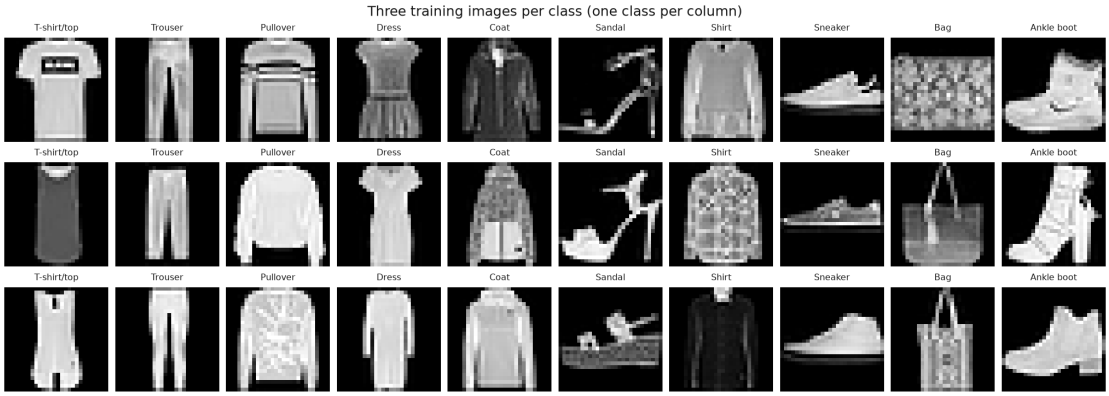


Figure 1: Three representative training images per class (one class per column) across the 10 Fashion-MNIST categories.

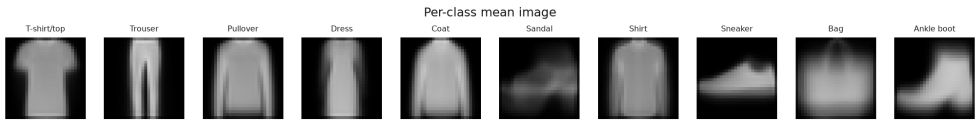


Figure 2: Per-class mean images across the 10 Fashion-MNIST categories. T-shirt/top, Pullover, Coat, and Shirt average to nearly the same torso silhouette, predicting the dominant confusion cluster.

1.4 Data Splitting

All six models in our benchmark share the exact same data partitions:

- **Split policy:** The official test set (10,000 images) is kept strictly untouched, while the official 60,000-image development set is partitioned into a 90%/10% train/validation split.
- **Random seed and stratification:** A deterministic pseudorandom generator initialized at seed 42 is used with class stratification enabled. The 6,000 development indices of each class are shuffled deterministically and partitioned into 5,400 training and 600 validation samples per class.
- **Exact sample counts:**
  - **Training split:** 54,000 images (5,400 per class).
  - **Validation split:** 6,000 images (600 per class).
  - **Test split:** 10,000 images (1,000 per class).
- **Split unit and leakage prevention:** The split unit is an individual image (Fashion-MNIST contains no multi-view subject or session groupings). The training procedure never accesses the test split; test images are evaluated only once after model selection on validation Macro-F1 is finalized. Furthermore, training and validation subsets are instantiated with independent transformation pipelines so that validation samples are never altered by training-time stochastic operations. The standardization constants ($\mu = 0.2860$, $\sigma = 0.3530$) are identical to four decimal places on both the 54,000-image training split and the 60,000-image development set, with the 10,000-image test set completely excluded.

1.5 Preprocessing and Input Representations

- **Data integrity:** All 70,000 samples contain complete 28×28 pixel grids and valid integer labels in $\{0, \ldots, 9\}$ with no missing or corrupted entries.
- **Normalization and augmentation policy:** Raw integer pixel intensities $X \in [0, 255]$ are linearly rescaled to $[0.0, 1.0]$ and standardized channel-wise using $\mu = 0.2860$ and $\sigma = 0.3530$:
$$\tilde{X} = \frac{X/255.0 - 0.2860}{0.3530}.$$
Although the data pipeline supports optional training-only random horizontal flipping and random 28×28 cropping with two-pixel padding, data augmentation is disabled across all six main benchmark runs so that the comparison isolates architectural inductive bias without confounding from random spatial cropping.
- **Input tensor shapes and empirical batch verification:**
  - Mini-batches of size $B = 128$ are emitted as 32-bit floating-point tensors of shape $(128, 1, 28, 28)$ paired with 64-bit integer label vectors of shape $(128,)$.

- Measured empirically on a preprocessed training batch, pixel activations exhibit a mean of 0.0073 and a standard deviation of 0.9989 (spanning $[-0.8102, +2.0227]$), confirming proper zero-mean unit-variance standardization.
- Each architecture family reshapes the standardized batch $(B, 1, 28, 28)$ into its corresponding structural view:
  1. **Linear and MLP:** Flattened into a one-dimensional feature vector $(B, 784)$.
  2. **CNN:** Retained as a two-dimensional spatial grid $(B, 1, 28, 28)$.
  3. **LSTM, GRU, and Transformer:** Reshaped into an ordered top-to-bottom sequence $(B, 28, 28)$ consisting of $T = 28$ horizontal row tokens, each of feature dimension $D_{\text{in}} = 28$. Vertical column serialization and 7×7 spatial patch tokenization are also supported for ablation studies.

2 Methodology

2.1 Pipeline Overview

Our end-to-end experimental workflow follows the nine-stage structure illustrated in Figure 3, progressing from raw image ingestion through representation construction, forward and loss computation, iterative parameter optimization, validation-guided checkpoint selection, and held-out test evaluation.

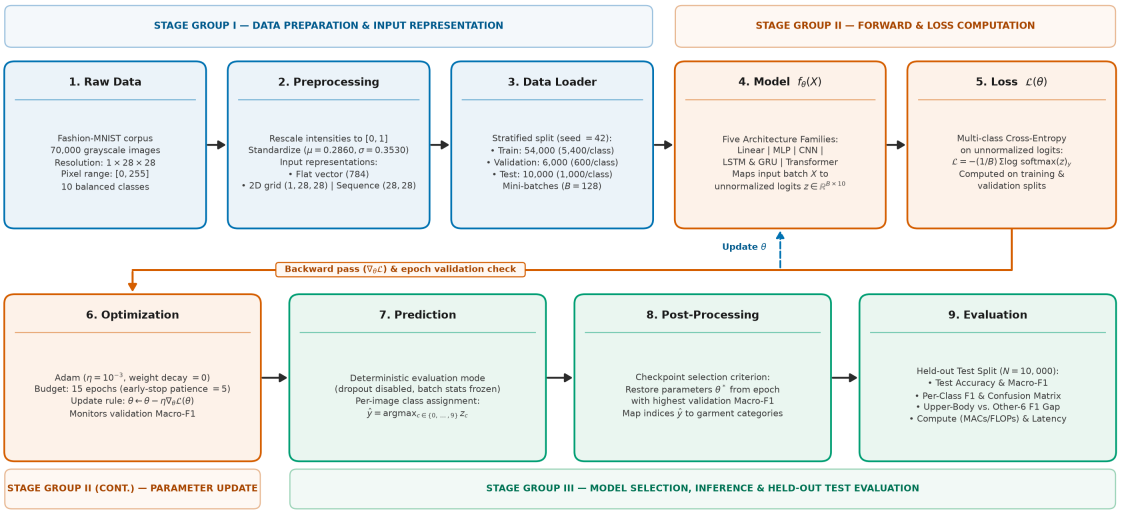


Figure 3: End-to-end experimental pipeline from raw Fashion-MNIST images

Specifically, the nine stages operate as follows:

1. **Raw data:** Ingestion of the 70,000 grayscale $1 \times 28 \times 28$ Fashion-MNIST images ($[0, 255]$ intensity range across 10 balanced apparel classes).
2. **Preprocessing:** Rescaling pixel intensities to $[0, 1]$, standardization via $\mu = 0.2860$ and $\sigma = 0.3530$, and conversion into one of three structural representations: flattened vector (784), 2D spatial grid $(1, 28, 28)$, or 28-step horizontal row sequence $(28, 28)$.

3. **Data loader:** Deterministic stratified partitioning (seed 42) into 54,000 training, 6,000 validation, and 10,000 held-out test samples, served in mini-batches of size $B = 128$.

4. **Model (f_θ):** Forward propagation through one of the six self-implemented neural architectures (Linear, MLP, CNN, LSTM, GRU, or Transformer) to produce unnormalized class logits $z \in \mathbb{R}^{B\times 10}$.

5. **Loss ($\mathcal{L}(\theta)$):** Evaluation of the multi-class Cross-Entropy objective directly on unnormalized logits z and ground-truth labels y.

6. **Optimization:** Backpropagation of parameter gradients $\nabla_\theta \mathcal{L}$ and weight updates using Adam ($\eta = 10^{-3}$, zero weight decay) over a 15-epoch schedule, with validation Macro-F1 tracked after every epoch.

7. **Prediction:** Deterministic forward evaluation, with dropout disabled and batch-normalization statistics frozen, assigning each image to $\hat{y} = \arg\max_{c\in\{0,\dots,9\}} z_c$.

8. **Post-processing:** Restoration of the optimal parameter state θ^* from the epoch with the highest validation Macro-F1, followed by index-to-category mapping.

9. **Evaluation:** Assessment on the held-out 10,000-image test set across overall Accuracy, Macro-F1, Per-Class F1, the Upper-Body vs. Other-6 F1 gap, row-normalized confusion matrices, analytical compute (MACs/FLOPs), and inference latency.

2.2 Baseline and Main Methods

All six models are self-implemented from foundational neural-network layers without pretrained weights or external vision backbones. Every architecture emits raw, unnormalized logits $z \in \mathbb{R}^{B\times 10}$ directly to the Cross-Entropy loss function, without an intermediate softmax layer during training.

2.2.1 Linear / Softmax Classifier (Baseline)

- **Input representation:** Flattened pixel vector $x \in \mathbb{R}^{B\times 784}$.
- **Architecture:** A single affine transformation mapping $\mathbb{R}^{784} \to \mathbb{R}^{10}$ via $z = xW^\top + b$, with weight matrix $W \in \mathbb{R}^{10\times 784}$ and bias $b \in \mathbb{R}^{10}$.
- **Parameter count and compute:** 7,850 trainable parameters; 7,840 MACs (approximately 0.008 MMACs or 15.7 KFLOPs) per image.
- **Strengths and limits:** Convex optimization landscape and minimal forward latency (0.0006 ms/image on GPU), establishing a transparent linear lower bound. However, its decision boundaries are linear hyperplanes in raw pixel space that cannot capture pairwise pixel interactions or two-dimensional spatial locality.

2.2.2 Multilayer Perceptron (MLP)

- **Input representation:** Flattened pixel vector $x \in \mathbb{R}^{B\times 784}$.
- **Architecture:** A two-layer feed-forward network with a 256-unit hidden layer and dropout probability $p = 0.2$:

  $$\text{Flatten}(784) \longrightarrow \text{Linear}(784 \to 256) \longrightarrow \text{ReLU} \longrightarrow \text{Dropout}(p = 0.2) \longrightarrow \text{Linear}(256 \to 10).$$

- **Activation and regularization rationale:** Without a nonlinear activation, stacked affine layers collapse algebraically into a single linear transformation equivalent to the baseline. The Rectified Linear Unit, $\text{ReLU}(u) = \max(0, u)$, enables piecewise-linear feature composition while maintaining a unit derivative on active units to prevent gradient saturation. Dropout ($p = 0.2$) randomly zeroes 20% of hidden activations during each training step, discouraging brittle co-adaptation among hidden neurons, and is disabled during evaluation.
- **Parameter count and compute:** 203,530 trainable parameters; 203,264 MACs (approximately 0.20 MMACs or 0.41 MFLOPs) per image.
- **Strengths and limits:** Captures global nonlinear combinations of pixel intensities, but because the input is flattened, the network has no structural prior that adjacent coordinates form two-dimensional neighborhoods.

2.2.3 Convolutional Neural Network (CNN)

- **Input representation:** Two-dimensional spatial grid $X \in \mathbb{R}^{B\times 1\times 28\times 28}$.
- **Architecture:** A self-designed two-stage convolutional feature extractor (32 and 64 channels) followed by a linear classification head:
  - **Stage 1:** 3×3 convolution ($1 \to 32$ channels, unit zero-padding) $\to$ 2D Batch Normalization $\to$ ReLU $\to$ 2×2 Max Pooling ($28 \times 28 \to 14 \times 14$).
  - **Stage 2:** 3×3 convolution ($32 \to 64$ channels, unit zero-padding) $\to$ 2D Batch Normalization $\to$ ReLU $\to$ 2×2 Max Pooling ($14 \times 14 \to 7 \times 7$).
  - **Classification head:** Spatial flattening ($64 \times 7 \times 7 = 3{,}136$) $\to$ Dropout ($p = 0.2$) $\to$ linear projection ($3{,}136 \to 10$).
- **Convolution, feature maps, and pooling mechanisms:**
  - *Convolution:* Learned 3×3 kernels slide across the 2D image plane to compute localized weighted sums. Because each kernel shares its weights across all spatial positions (i, j), local motifs such as collar edges, sleeve seams, and heel contours are detected translation-equivariantly using only $9 \times C_{\text{in}}$ weights per output channel. Unit padding preserves spatial resolution prior to pooling.
  - *Feature maps and normalization:* Each kernel emits a 2D activation map quantifying the presence of its learned motif across space. Stage 1 extracts 32 low-level edge and gradient maps, whereas Stage 2 combines all 32 maps to form 64 higher-order texture and garment-part representations. Batch normalization stabilizes channel-wise activation distributions throughout training.

  - *Spatial pooling:* 2×2 max pooling retains the peak response within each non-overlapping 2×2 window, halving spatial dimensions at each stage ($28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$). This provides local translation invariance, reduces intermediate activation memory by $4\times$ per stage, and expands the effective receptive field of subsequent 3×3 filters.

- **Parameter count and compute:** 50,378 trainable parameters ($4.0\times$ fewer than the MLP); 3,869,824 MACs (approximately 3.87 MMACs or 7.74 MFLOPs) per image.
- **Strengths and limits:** Encodes 2D spatial locality and translation equivariance directly into the architecture, achieving the strongest accuracy across all garment classes with a compact parameter footprint.

2.2.4 Recurrent Neural Networks (LSTM and GRU)

- **Input representation:** Each image is serialized top-to-bottom into an ordered sequence of $T = 28$ horizontal rows, $X_{\text{seq}} \in \mathbb{R}^{B\times 28\times 28}$.
  - **Timestep ($T = 28$):** Each step $t \in \{1, \ldots, 28\}$ represents one horizontal scanline progressing from the top ($t = 1$, neckline/collar region) to the bottom ($t = 28$, hem/sole region).
  - **Token dimension ($D_{\text{in}} = 28$):** Each input vector $x_t \in \mathbb{R}^{28}$ contains the 28 horizontal pixel intensities of row t.
- **Architecture and recurrent state dynamics:**
  - A single-layer unidirectional recurrent network with hidden dimension $H = 128$ sequentially updates a hidden state $h_t \in \mathbb{R}^{128}$ from (x_t, h_{t-1}) across $t = 1, \ldots, 28$.
  - **GRU:** Employs two gating vectors (update gate and reset gate) to interpolate between the previous state h_{t-1} and a candidate recurrent activation without maintaining a separate memory cell.
  - **LSTM:** Maintains a dedicated cell state $c_t \in \mathbb{R}^{128}$ regulated by three gates (input, forget, and output) to propagate long-range sequential information along an additive memory pathway.
  - **Classification readout:** The terminal hidden state $h_{28} \in \mathbb{R}^{B\times 128}$, summarizing the complete top-to-bottom scan, passes through Dropout ($p = 0.2$) and a linear projection ($\mathbb{R}^{128} \rightarrow \mathbb{R}^{10}$).
- **Parameter counts and compute:**
  - **GRU:** 61,962 trainable parameters; 1,678,592 MACs (approximately 1.68 MMACs or 3.36 MFLOPs) per image.
  - **LSTM:** 82,186 trainable parameters; 2,237,696 MACs (approximately 2.24 MMACs or 4.48 MFLOPs) per image.
- **Strengths and limits:** Preserves horizontal within-row structure and shares recurrent transition weights across vertical scan steps, achieving high parameter efficiency. However, vertical pixel adjacency is compressed into sequential state transitions, requiring visual cues from the neckline ($t \approx 1$–5) to survive more than 20 recurrent updates before reaching the terminal classifier at $t = 28$.

2.2.5 Transformer Encoder Classifier

- **Input representation:** The same ordered sequence of $T = 28$ horizontal row tokens, $X_{\text{seq}} \in \mathbb{R}^{B\times 28\times 28}$.
- **Token projection, classification token, and positional encoding:**
  - *Row embedding projection:* A shared linear layer projects each 28-dimensional row vector into model dimension $d_{\text{model}} = 128$.
  - *Classification token ([CLS]):* A learnable embedding vector of dimension 128 is prepended to the 28 row embeddings, forming a sequence of length $T + 1 = 29$.
  - *Positional encoding:* Because self-attention is inherently permutation-equivariant (permuting the image rows would otherwise leave the [CLS] representation unchanged), a learnable positional embedding tensor of shape $(1, 29, 128)$ is added element-wise to the token sequence before it enters the encoder stack.
- **Self-attention mechanism and readout:**
  - The token sequence traverses $L = 2$ pre-norm Transformer encoder layers ($d_{\text{model}} = 128$, $H = 4$ attention heads of width $d_k = 32$ each, feed-forward dimension $d_{\text{ff}} = 256$ with GELU nonlinearity, and dropout $p = 0.2$).
  - Within each layer, multi-head self-attention projects the normalized token sequence into queries Q, keys K, and values V, computing scaled dot-product attention outputs $\text{softmax}(QK^\top/\sqrt{32})V$. Every row token can therefore attend directly to every other row token in $O(1)$ path length from the first layer.
  - After the second encoder layer, the output representation at the prepended [CLS] index undergoes Layer Normalization and a linear projection ($\mathbb{R}^{128} \to \mathbb{R}^{10}$).
- **Parameter count and compute:** 274,058 trainable parameters; 8,134,400 MACs (approximately 8.13 MMACs or 16.27 MFLOPs) per image.
- **Strengths and limits:** Provides an immediate global receptive field across all image rows without sequential bottlenecks, but lacks built-in spatial locality priors, requiring positional relationships and local row-to-row continuity to be learned entirely from data.

2.3 Training Setup

To isolate the effects of input representation and architectural inductive bias, all six models are trained under an identical experimental protocol:

- **Optimizer and learning rate:** Adam with constant learning rate $\eta = 10^{-3}$, weight decay $\lambda = 0.0$, and no learning-rate scheduler.
- **Mini-batch size and epoch budget:** Mini-batch size $B = 128$ over a fixed budget of 15 epochs.
- **Regularization:** Dropout with probability $p = 0.2$ in all nonlinear architectures; 2D Batch Normalization in the CNN; pre-layer Layer Normalization in the Transformer.

- **Early stopping and checkpoint selection:** Validation Macro-F1 is evaluated at the conclusion of every epoch with an early-stopping patience of 5 epochs (early stopping was never triggered; all six models completed the full 15 epochs). Whenever validation Macro-F1 improves, the corresponding model parameters θ^* are saved as the best checkpoint, which is subsequently restored for final test evaluation.
- **Reproducibility and determinism:** A global random seed of 42 governs dataset splitting, weight initialization, mini-batch shuffling, and dropout masks, with deterministic CUDA convolution algorithms enforced across all runs.
- **Hardware and software environment:**
  - **Hardware:** NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM, CUDA 13.0) on Windows 11.
  - **Software:** Python 3.13.14, PyTorch 2.14.0, torchvision 0.29.0, NumPy 2.5.3, and scikit-learn 1.9.1 using standard 32-bit floating-point precision (FP32). All six benchmark runs were executed back-to-back in a single controlled session.

2.4 Experimental Design

- **Experiment 1 — Five-Family Architectural and Representation Benchmark (Linear, MLP, CNN, LSTM/GRU, Transformer):**
  - **Hypothesis (H_1):** Classification performance on Fashion-MNIST is governed primarily by how well an architecture's input representation and inductive bias preserve two-dimensional spatial structure (2D Grid CNN > 1D Row Sequence RNN/Transfo 1D Flattened MLP/Linear) rather than raw parameter count. Inter-model performance differences are expected to concentrate within the four visually overlapping upper-body classes (Shirt, T-shirt/top, Pullover, and Coat).
  - **Manipulated factor:** Model architecture family and input representation ($(B, 784)$ flattened vector vs. $(B, 1, 28, 28)$ 2D spatial grid vs. $(B, 28, 28)$ horizontal row sequence).
  - **Controlled factors:** Dataset (Fashion-MNIST), stratified split (54,000/6,000/10,000 at seed 42), pixel standardization ($\mu = 0.2860$, $\sigma = 0.3530$), Cross-Entropy loss, Adam optimizer ($\eta = 10^{-3}$, zero weight decay), batch size 128, epoch budget 15, and validation Macro-F1 checkpoint selection rule.
  - **Evaluation metrics and three-tier single-seed decision criteria:** We evaluate Test Accuracy, Test Macro-F1, Per-Class F1 (specifically the F1 gap between the four upper-body classes and the remaining six classes), trainable parameter count, analytical compute (MACs/FLOPs), training wall-clock time, and GPU versus end-to-end inference latency. Because all benchmark runs use a single random seed (42), pairwise Test Macro-F1 differences (Δ) are interpreted using three non-overlapping tiers:
    1. $\Delta < 0.01$ **(practically tied or inconclusive under single-seed evaluation):** Differences below 1.0 percentage point (such as MLP 0.8902 vs. LSTM 0.8898 at $\Delta = 0.0004$, and LSTM 0.8898 vs. Transformer 0.8825 at $\Delta = 0.0073$) lie within single-seed stochastic variation and do not support superiority claims without multi-seed variance estimates.

2. $0.01 \leq \Delta \leq 0.02$ **(moderate empirical advantage when corroborated):** Margins between 1.0 and 2.0 percentage points—specifically CNN (0.9163) vs. GRU (0.9010) ($\Delta = +0.0153$, where the CNN leads across all 10 classes with 18.7% fewer parameters) and GRU (0.9010) vs. MLP (0.8902) ($\Delta = +0.0108$, where the GRU leads on 8 of 10 classes with 3.3× fewer parameters)—represent meaningful empirical advantages in this experiment, though multi-seed runs remain desirable for formal confidence intervals.
3. $\Delta > 0.02$ **(large, decisive architectural separation):** Margins exceeding 2.0 percentage points (such as CNN 0.9163 vs. MLP 0.8902 at +0.0261, CNN vs. Transformer at +0.0338, and all nonlinear architectures vs. Linear 0.8441 at +0.0384 to +0.0722) reflect clear structural separations.

- **Experiment 2 — Optional Extension (Section 11.2): Controlled LSTM vs. GRU Comparison on Row Sequences:**
  - **Hypothesis (H_2):** Over a short 28-step horizontal row sequence with matched hidden width ($H = 128$) and depth (one layer), the two-gate GRU will match or exceed the three-gate LSTM in test Macro-F1 while requiring approximately 25% fewer parameters and FLOPs.
  - **Manipulated factor:** Recurrent gating cell formulation (LSTM vs. GRU).
  - **Controlled factors:** Row sequence representation (28 steps × 28 features), hidden dimension 128, layer depth 1, dropout $p = 0.2$, linear readout head (128 → 10), and all optimization hyperparameters.

3 Implementation Results

3.1 Training Behavior

Figure 4 presents the 15-epoch training and validation Cross-Entropy Loss and Macro-F1 trajectories across all six architectures.

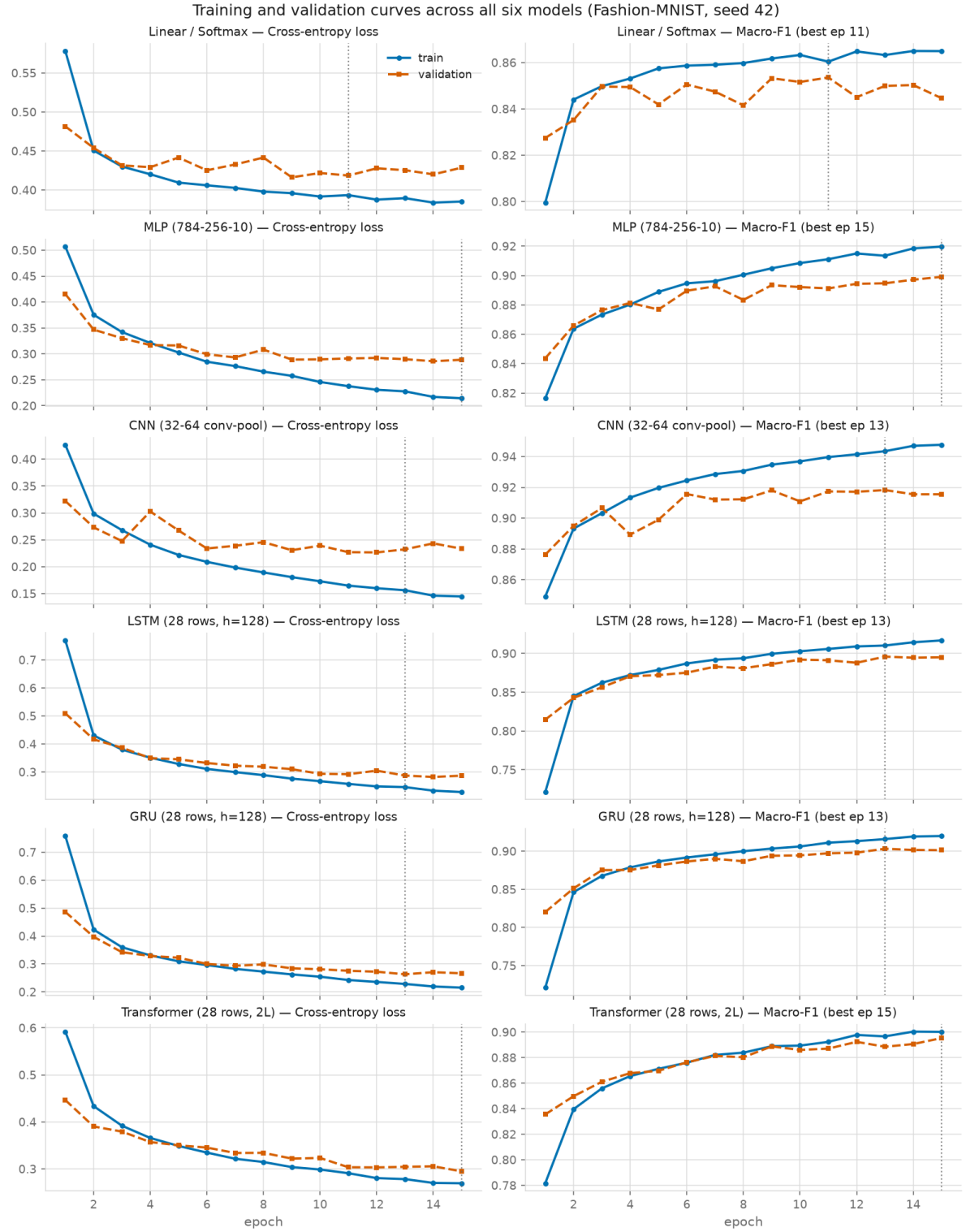


Figure 4: Training and validation Cross-Entropy Loss (left column) and Macro-F1 (right column) over 15 epochs for all six architectures on Fashion-MNIST (seed 42). Vertical dotted lines mark the best validation Macro-F1 checkpoint selected for test evaluation.

- **Convergence dynamics, underfitting versus overfitting, and selected checkpoints:**
  - **Linear:** Exhibits high-bias structural underfitting relative to nonlinear models, plateauing near 0.385 training loss and 0.865 training Macro-F1 because a

linear hyperplane cannot partition curved feature manifolds. It also shows mild late-epoch overfitting after epoch 11/15: validation Macro-F1 peaks at epoch 11 (0.8536, with validation loss 0.4186 and training loss 0.3935), after which training loss continues down to 0.3853 at epoch 15 while validation loss rises to 0.4287 and validation Macro-F1 falls to 0.8446. Restoring the epoch-11 checkpoint avoids this late degradation.

- **MLP:** Improves monotonically across the schedule and reaches peak validation Macro-F1 at epoch 15/15 (0.8991 validation Macro-F1, 0.2887 validation loss, 0.2145 training loss, and 0.9197 training Macro-F1). Since validation performance is still trending upward at epoch 15, the MLP has not fully saturated its epoch budget.
- **CNN:** Converges most rapidly, surpassing 0.90 validation Macro-F1 by epoch 3 (0.9066) and achieving its optimal checkpoint at epoch 13/15 (0.9183 validation Macro-F1, 0.9185 validation accuracy, 0.2323 validation loss, and 0.1562 training loss). During epochs 14–15, training loss declines further to 0.1447 while validation Macro-F1 dips slightly to 0.9155; checkpoint selection avoids this late decline.
- **LSTM and GRU:** Both recurrent architectures peak at epoch 13/15 and stabilize across epochs 13–15. LSTM reaches 0.8959 validation Macro-F1 and 0.8970 validation accuracy; GRU reaches 0.9032 validation Macro-F1 and 0.9038 validation accuracy.
- **Transformer:** Selects its final epoch, 15/15 (0.8953 validation Macro-F1, 0.8970 validation accuracy, 0.2949 validation loss, and 0.2695 training loss). Both training and validation losses continue decreasing through epoch 15, indicating that this 274K-parameter self-attention encoder has not fully converged under the short, constant-learning-rate schedule without warmup.

- **Methodological and engineering refinements during implementation:**

1. *Eliminating non-deterministic GPU convolution variance:* Fixing standard pseudorandom seeds reproduced the Linear, MLP, LSTM, GRU, and Transformer runs exactly, whereas runtime convolution algorithm autotuning on the GPU initially caused the CNN's peak validation Macro-F1 to fluctuate slightly between epochs 9 and 13 across repeated runs. Enforcing deterministic cuDNN convolution kernels made the CNN trajectory bit-for-bit reproducible, verified across repeated runs matching training loss ($0.4268 \rightarrow 0.2987 \rightarrow 0.2680$) and validation Macro-F1 ($0.8763 \rightarrow 0.8950 \rightarrow 0.9066$) across the first three epochs.
2. *Decoupling model inference latency from host data-loading overhead:* Timing an evaluation loop directly over a single-process data loader conflates CPU image decoding and standardization (approximately 0.15–0.20 ms/image) with GPU forward execution. We therefore report two distinct metrics: **model-only inference latency**, measured on preloaded GPU tensors with CUDA synchronization, and **end-to-end inference latency**, including host-to-device data loading.

3.2 Quantitative Results

Table 2 summarizes the primary and secondary metrics on the held-out 10,000-image official test set (seed 42), including trainable parameter counts, analytical forward-pass compute per image, training wall-clock time, and both inference latency measurements. Table 3 reports the per-class F1 scores across all 10 garment categories.

Table 2: Overall quantitative comparison on the Fashion-MNIST held-out test set ($N = 10{,}000$, seed 42).

Model	Input representation	Params	Compute/image (MMACs/MFLOPs)	Best ep.	Val Macro-F1 (Val Acc.)	Test Acc.	Test Macro-F1	Train time	Model-only (ms/img)	End-to-end (ms/img)
Linear / Softmax	Flat vector (784)	7,850	0.008 / 0.016	11/15	0.8536 (0.8533)	0.8440	0.8441	125.2 s (8.3 s/ep)	0.0006	0.1992
MLP (784–256–10)	Flat vector (784)	203,530	0.20 / 0.41	15/15	0.8991 (0.8998)	0.8909	0.8902	137.6 s (9.2 s/ep)	0.0015	0.1816
CNN (32–64 Conv-Pool)	2D grid $(1, 28, 28)$	50,378	3.87 / 7.74	13/15	**0.9183 (0.9185)**	**0.9165**	**0.9163**	253.6 s (16.9 s/ep)	0.0096	0.1712
LSTM (28 rows, $H = 128$)	28 rows × 28 px	82,186	2.24 / 4.48	13/15	0.8959 (0.8970)	0.8906	0.8898	200.7 s (13.4 s/ep)	0.0085	0.1538
GRU (28 rows, $H = 128$)	28 rows × 28 px	61,962	1.68 / 3.36	13/15	0.9032 (0.9038)	0.9017	0.9010	218.9 s (14.6 s/ep)	0.0050	**0.1465**
Transformer (28 rows, $d = 128$)	28 row tokens + [CLS]	274,058	8.13 / 16.27	15/15	0.8953 (0.8970)	0.8846	0.8825	259.2 s (17.3 s/ep)	0.0151	0.1549

Single-seed interpretation (seed 42): Test Macro-F1 ranks the models as CNN (0.9163) > GRU (0.9010) > MLP (0.8902) ≈ LSTM (0.8898) > Transformer (0.8825) > Linear (0.8441). Under the three-tier criteria in Section 2.4, margins above 0.02 (CNN over MLP, LSTM, Transformer, and Linear; and all nonlinear models over Linear) are large empirical separations. Margins in $[0.01, 0.02]$ (CNN over GRU by $+0.0153$ with fewer parameters, and GRU over MLP by $+0.0108$ with $3.3\times$ fewer parameters) are moderate empirical advantages. Margins below 0.01 (MLP versus LSTM at 0.0004, and LSTM versus Transformer at 0.0073) are treated as practically tied under this single-seed criterion; these comparisons are not significance tests.

Table 3: Per-class test F1 scores on Fashion-MNIST (seed 42), ordered by Linear baseline difficulty.

Class	Linear	MLP	CNN	LSTM	GRU	Transformer
Shirt (6)	0.5999	0.7138	**0.7586**	0.7024	0.7220	0.6636
Pullover (2)	0.7323	0.8091	**0.8701**	0.8235	0.8431	0.8129
Coat (4)	0.7442	0.8236	**0.8659**	0.8229	0.8419	0.8094
T-shirt/top (0)	0.8073	0.8430	**0.8789**	0.8326	0.8503	0.8343
Dress (3)	0.8492	0.8870	**0.9117**	0.8905	0.9004	0.8777
Sneaker (7)	0.9238	0.9509	**0.9587**	0.9439	0.9575	0.9491
Sandal (5)	0.9314	0.9641	**0.9762**	0.9619	0.9759	0.9670
Bag (8)	0.9355	0.9716	**0.9814**	0.9790	0.9755	0.9692
Ankle boot (9)	0.9499	0.9594	**0.9720**	0.9599	0.9630	0.9630
Trouser (1)	0.9672	0.9800	**0.9894**	0.9814	0.9804	0.9784
Mean F1, Upper-Body 4 $(0, 2, 4, 6)$	0.7209	0.7974	**0.8434**	0.7954	0.8143	0.7801
Mean F1, Other 6 $(1, 3, 5, 7, 8, 9)$	0.9261	0.9522	**0.9649**	0.9528	0.9588	0.9507
Gap (Other 6 − Upper-Body 4)	0.2052	0.1548	**0.1215**	0.1574	0.1444	0.1707
Macro-F1 (all 10 classes)	0.8441	0.8902	**0.9163**	0.8898	0.9010	0.8825

3.3 Qualitative Results and Error Analysis

Figures 5 and 6 show representative misclassified and correctly classified test examples from the CNN, while Figure 7 presents row-normalized test confusion matrices for all six

architectures. Table 4 quantifies exact test error counts and relative error shares across the principal confusion clusters.

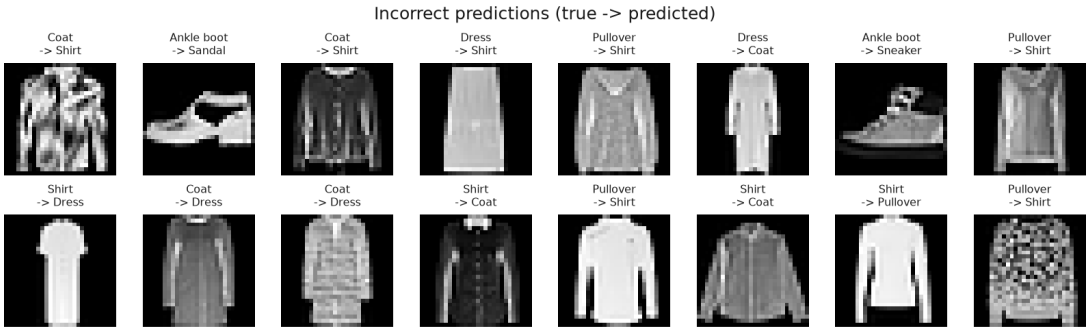


Figure 5: CNN misclassifications on the test split (true and predicted category with confidence), showing errors concentrated among visually overlapping upper-body garments.

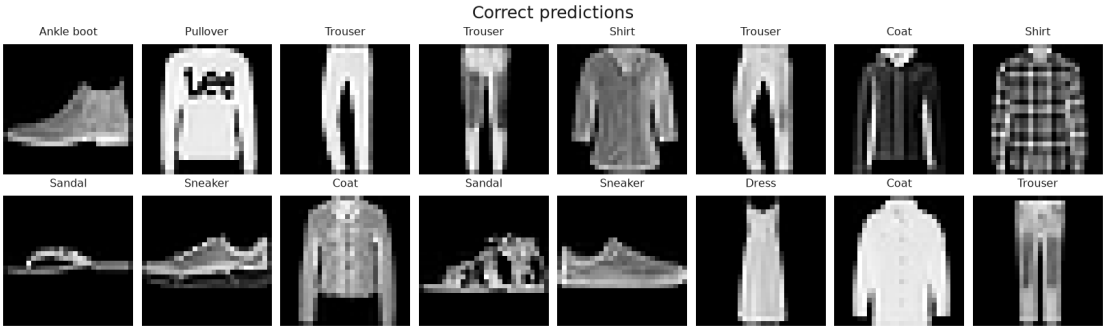


Figure 6: Correct CNN test examples, including clean-silhouette items (Trouser, Bag, Sandal) and challenging long-sleeved upper-body garments (Shirt, Pullover, Coat).

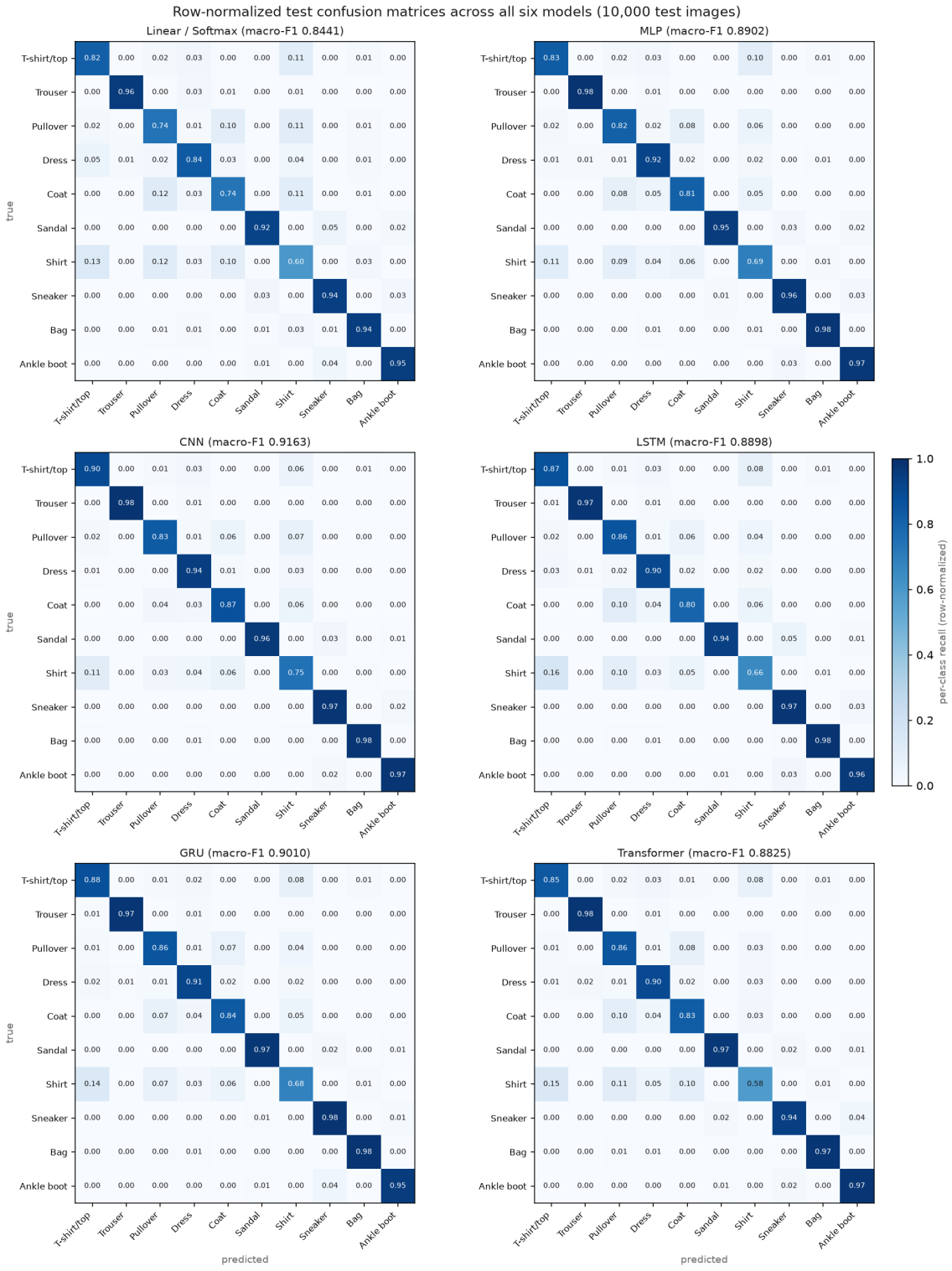


Figure 7: Row-normalized confusion matrices on the 10,000-image Fashion-MNIST test split across Linear, MLP, CNN, LSTM, GRU, and Transformer.

Table 4: Error-group frequencies across all six models on the official test split ($N = 10{,}000$; percentages are shares of each model's total errors).

Error group (true → predicted)	Linear	MLP	CNN	LSTM	GRU	Transformer
Total test errors	1,560	1,091	**835**	1,094	983	1,154
Group 1a: Shirt → T-shirt/top	134 (8.6%)	**106** (9.7%)	113 (13.5%)	161 (14.7%)	137 (13.9%)	146 (12.7%)
Group 1b: T-shirt/top → Shirt	109 (7.0%)	105 (9.6%)	**56** (6.7%)	79 (7.2%)	82 (8.3%)	77 (6.7%)
Group 2: Pullover ↔ Coat	223 (14.3%)	165 (15.1%)	**95** (11.4%)	161 (14.7%)	140 (14.2%)	177 (15.3%)
Group 3: Shirt → Pullover / Coat	211 (13.5%)	143 (13.1%)	**97** (33+64, 11.6%)	150 (97+53, 13.7%)	137 (75+62, 13.9%)	213 (113+100, 18.5%)
All confusions inside upper-body quartet $\{0, 2, 4, 6\}$	935 (59.9%)	674 (61.8%)	**528** (63.2%)	691 (63.2%)	620 (63.1%)	722 (62.6%)
Group 4: Footwear internal $\{5, 7, 9\}$	182 (11.7%)	120 (11.0%)	**89** (10.7%)	130 (11.9%)	98 (10.0%)	113 (9.8%)
Group 5: Cross-group apparel/footwear/bag	121 (7.8%)	53 (4.9%)	**32** (3.8%)	38 (3.5%)	49 (5.0%)	58 (5.0%)

3.3.1 Detailed Analysis of Error Groups

1. **Group 1 — Bidirectional Shirt ↔ T-shirt/top boundary (169 CNN errors, up to 243 Linear and 240 LSTM errors):**

   - *Description and examples:* Short-sleeved or collarless Shirt instances are misclassified as T-shirt/top (e.g., Figure 5, true Shirt predicted as T-shirt/top with confidence $p = 0.53$), and boxy T-shirt/top instances are misclassified as Shirt.
   - *Frequency:* Shirt → T-shirt/top errors remain in a narrow band of 106–161 test images across all six architectures. Moving from Linear to CNN reduces total test errors by 46.5% (1,560 → 835), but reduces Shirt → T-shirt/top errors by only 15.7% (134 → 113).
   - *Hypothesized cause:* This persistent error floor reflects intrinsic visual ambiguity at 28×28 grayscale resolution: fine structural cues separating a casual short-sleeved shirt from a T-shirt—a collar seam or vertical button placket only 1–2 pixels wide—frequently vanish during downsampling.
   - *Concrete improvements:* Use higher-resolution source imagery when available, or introduce localized contrast normalization and multi-scale (1×1 and 3×3) convolutional branches to amplify subtle vertical placket gradients.

2. **Groups 2 and 3 — Long-sleeved upper-body confusions (Pullover ↔ Coat and Shirt → Pullover/Coat):**

   - *Description and examples:* Long-sleeved Shirt, Pullover, and Coat items share nearly identical torso-and-sleeve contours (e.g., Figure 5, true Shirt predicted as Pullover with $p = 0.49$ and as Coat with $p = 0.92$; correctly resolved boundary cases in Figure 6 exhibit visible neckline V-openings or hemline splits).
   - *Frequency:* CNN reduces Pullover ↔ Coat confusions by 57.4% relative to Linear (223 → 95), and Shirt → Pullover/Coat confusions by 54.0% (211 → 97, comprising 33 Pullover and 64 Coat predictions). Conversely, the row-tokenized Transformer records the highest Shirt → Pullover/Coat count (213 = 113 Pullover +100 Coat predictions, or 18.5% of its test errors).
   - *Hypothesized cause:* Distinguishing a knit pullover from a buttoned shirt or open-front coat depends on local 2D surface micro-textures (knit ribbing, vertical lapel/zipper lines, and collar openings) spanning 3×3 pixel neighborhoods. A 1D horizontal row-token representation separates some vertical neighborhoods across timesteps.
   - *Concrete improvements:* (i) Replace row tokens with 7×7 two-dimensional spatial patch tokens to preserve local neighborhoods; (ii) extend Transformer

and MLP schedules to 30–50 epochs with linear learning-rate warmup and cosine annealing; and (iii) investigate a two-stage hierarchical classifier that routes upper-body predictions to a dedicated fine-grained discriminator.

3. **Groups 4 and 5 — Internal footwear confusions and cross-group outliers:**
   - *Description and frequency:* Internal confusions among Sandal, Sneaker, and Ankle boot decline from 182 (Linear) to 89 (CNN), occurring primarily between low-cut ankle boots and high-top sneakers (22 Sneaker → Ankle boot and 23 Ankle boot → Sneaker errors in CNN). Cross-group confusions (apparel versus footwear or bags, such as narrow vertical handbags misclassified as Dress in Figure 5) drop by 73.6% from Linear (121) to CNN (32).
   - *Hypothesized cause and improvement:* Minor vertical framing offsets alter the apparent shaft-to-sole aspect ratio of footwear and tall bags. Training-time spatial augmentation (random horizontal flips and two-pixel-padded random crops) could improve translation tolerance along object boundaries.

3.4 Comparison and Discussion

1. **Evaluation of Hypothesis H_1 (partially supported):**
   - **Supported:** CNN ranks first across aggregate and per-class metrics (0.9163 Macro-F1, leading all 10 classes in Table 3) while using 4.0× fewer parameters than MLP and 5.4× fewer than Transformer. Linear ranks last (0.8441), and row-recurrent GRU (0.9010) exceeds flattened MLP (0.8902) by 0.0108 Macro-F1 with 3.3× fewer parameters (61,962 versus 203,530). The performance spread among the five nonlinear architectures is 4.5× wider on the upper-body four (0.0633: 0.7801–0.8434) than on the other six classes (0.0142: 0.9507–0.9649).
   - **Nuanced or refuted aspect:** Preserving 1D horizontal row structure does not automatically guarantee superiority over flattened inputs. LSTM (0.8898 Macro-F1) is virtually tied with MLP (0.8902), while the row-token Transformer (0.8825) trails the MLP and both recurrent models under this 15-epoch budget.
2. **Impact of data representation and inductive bias:**
   - *2D spatial grid (CNN, 0.9163 Macro-F1):* Preserves both spatial axes and shares 3×3 filters across the image plane. This is consistent with 2D locality and translation equivariance providing useful priors on this 54,000-image dataset, reducing upper-body quartet confusions from 674 (MLP) and 722 (Transformer) to 528.
   - *1D row sequence (GRU 0.9010, LSTM 0.8898, Transformer 0.8825):* Retains horizontal adjacency within each row while encoding vertical structure through sequential transitions or positional embeddings. GRU and LSTM use fewer parameters than MLP, but row-sequence processing does not guarantee higher accuracy when vertical evidence must propagate across recurrent steps or be learned without local priors.
   - *Flattened vector (Linear 0.8441, MLP 0.8902):* Discards explicit 2D topology. Linear underfits, while a single 256-unit ReLU hidden layer allows MLP to reach 0.8902 Macro-F1 on centered, bounding-box-normalized thumbnails.

3. **Accuracy, latency, parameter, and compute trade-offs:**

   - **Model-only GPU latency and analytical compute:** With input tensors resident in GPU memory, measured forward latency follows this order in the benchmark: Linear (0.008 MMACs, 0.0006 ms) < MLP (0.20, 0.0015 ms) < GRU (1.68, 0.0050 ms) < LSTM (2.24, 0.0085 ms) < CNN (3.87, 0.0096 ms) < Transformer (8.13, 0.0151 ms). This is an observed ordering for the stated hardware and timing procedure.
   - **End-to-end latency:** Single-process CPU image decoding and standardization contribute approximately 0.15–0.20 ms/image, much longer than GPU model-only forward passes, so measured end-to-end latencies fall in a narrower 0.1465–0.1992 ms/image range.
   - **Deployment choices:** CNN is the accuracy choice (0.9163 Macro-F1) with 50,378 parameters; GRU offers a sequential accuracy–latency balance (0.9010 at 0.0050 ms/image); MLP delivers 0.8902 Macro-F1 with 6.4× lower GPU latency (0.0015 ms/image) and 1.8× shorter training time (137.6 s) than CNN; Linear has the smallest footprint (7,850 parameters) and fastest model-only inference (0.0006 ms/image) for highly constrained deployment.

3.5 Optional Extension (Section 11.2): LSTM vs. GRU Comparison

Table 5 compares LSTM and GRU under identical row tokenization (28 steps × 28 pixels), hidden dimension (128), layer depth (one), dropout (0.2), and Adam schedule (seed 42).

Table 5: Controlled recurrent comparison: LSTM versus GRU on Fashion-MNIST (seed 42).

Metric	LSTM (1 layer, $H = 128$)	GRU (1 layer, $H = 128$)	Comparison (GRU vs. LSTM)
Trainable parameters	82,186	**61,962**	GRU uses 20,224 fewer (24.6%)
Forward compute (MMACs/MFLOPs)	2.24 / 4.48	**1.68 / 3.36**	GRU requires 25.0% fewer MACs/FLOPs
Total training time	**200.7 s** (13.4 s/epoch)	218.9 s (14.6 s/epoch)	LSTM is 18.2 s (8.3%) faster
Model-only inference	0.0085 ms/image	**0.0050 ms/image**	GRU is 41.2% faster
End-to-end inference	0.1538 ms/image	**0.1465 ms/image**	GRU is 4.7% faster
Training loss (epoch 1 → 13 → 15)	0.7697 → 0.2460 → 0.2284	**0.7579 → 0.2286 → 0.2153**	GRU lower at all 15 epochs
Validation loss (epoch 1 → 13 → 15)	0.5093 → 0.2872 → 0.2868	**0.4863 → 0.2632 → 0.2666**	GRU lower at all 15 epochs
Training Macro-F1 (epoch 1 → 13 → 15)	0.7212 → 0.9102 → 0.9168	0.7210 → **0.9159 → 0.9200**	Tied at epoch 1; GRU higher at epochs 2–15
Validation Macro-F1 (epoch 1 → 13 → 15)	0.8146 → 0.8959 → 0.8951	**0.8202 → 0.9032 → 0.9014**	GRU higher at every epoch
Test Accuracy / Macro-F1	0.8906 / 0.8898	**0.9017 / 0.9010**	GRU leads by 0.0111 / 0.0112

- **Where GRU excels:** In this seed-42 comparison, GRU achieves 0.0112 higher test Macro-F1 (0.9010 versus 0.8898) and outperforms LSTM on 9 of 10 classes (Table 3), while requiring 24.6% fewer parameters, 25.0% fewer MACs, and 41.2% lower GPU forward-pass latency (0.0050 versus 0.0085 ms/image).
- **Where LSTM excels:** LSTM completes its 15 training epochs 18.2 s (8.3%) faster in wall-clock time (200.7 versus 218.9 s, or 13.4 versus 14.6 s/epoch), potentially reflecting fused recurrent backward-kernel scheduling and single-process host data-loading variance.
- **Why GRU achieved higher Macro-F1:** Epoch-by-epoch histories show lower GRU training and validation loss at every epoch and higher validation Macro-F1 at all 15 epochs. Training Macro-F1 is tied at epoch 1 and higher for GRU from epochs

2–15. This suggests faster optimization rather than LSTM overfitting; however, the explanation remains a hypothesis from this single run.

3.6 Limitations, Generalization, and Future Work

1. **Dataset limitations and generalization to real-world retail imagery:**
   - Fashion-MNIST thumbnails are low-resolution (28×28), single-channel grayscale, centered, and largely background-free (50.2% black pixels). The rankings observed here do not transfer unchanged to unconstrained retail photography, where object scale, camera viewpoint, illumination, and background clutter vary substantially. The relative CNN–Transformer result is specific to small, low-resolution images trained from scratch and may differ on large-scale, high-resolution benchmarks with pretraining.
   - Fine-grained categories such as Shirt versus T-shirt/top also exhibit intrinsic visual ambiguity at this resolution.
2. **Methodological and experimental limitations:**
   - **Single-seed evaluation (seed 42):** Without multi-seed replication and mean ± standard deviation, small margins (< 0.01, such as MLP versus LSTM) cannot establish a reliable ranking; moderate margins (0.01–0.02) also lack formal confidence intervals.
   - **Fixed 15-epoch budget without architecture-specific schedule tuning:** MLP and Transformer had not fully converged by epoch 15. The results therefore support the specific claim that CNN outperforms Transformer under an equal, short 15-epoch constant-learning-rate schedule without warmup, not a universal ranking across training horizons.
   - **Unmatched parameter budgets:** Parameter counts span a 35× range (7,850–274,058), so the comparison concerns concrete configurations rather than parameter-matched architecture families.
   - **Host-bound training times:** Single-process data loading mixes CPU preprocessing overhead into per-epoch wall-clock times.
3. **Future work and improvement roadmap:**
   - **Multi-seed replication:** Run seeds 42, 43, and 44 and report mean ± standard deviation across primary and per-class metrics.
   - **Patch tokenization and warmup for Transformer:** Evaluate 7×7 two-dimensional patch tokens over 30–50 epochs with linear warmup and cosine annealing.
   - **Parameter-matched and augmentation ablations:** Compare models near 60K parameters across families and separately evaluate training-time random horizontal flips and two-pixel-padded random crops.

3.7 Conclusion (Answer to the Research Question)

Across all five mandatory architecture families (Linear, MLP, CNN, LSTM/GRU, and Transformer) evaluated on a shared stratified Fashion-MNIST split (54,000/6,000/10,000,

seed 42) under an identical 15-epoch Adam protocol, test Macro-F1 ranges from 0.8441 (Linear) to 0.9163 (CNN). The results indicate that, in these configurations, classification performance depends strongly on how well the architecture preserves two-dimensional local image structure, rather than on parameter count alone. The two-stage CNN achieves the highest test accuracy (0.9165) and Macro-F1 (0.9163) across all 10 garment categories with 50,378 parameters (4.0× fewer than MLP and 5.4× fewer than Transformer). The row-recurrent GRU (0.9010 Macro-F1, 61,962 parameters) outperforms both LSTM (0.8898) and the flattened MLP (0.8902, 203,530 parameters). Moreover, the inter-model Macro-F1 spread is about 4.5× wider within the four silhouette-sharing upper-body classes than across the other six categories, consistent with localized 2D feature extraction being particularly useful for this task. These conclusions remain specific to a single seed and the tested training schedule.

4 Conclusion

Across all five mandatory architecture families (Linear, MLP, CNN, LSTM/GRU, and Transformer), evaluated on a shared stratified Fashion-MNIST split (54,000 training, 6,000 validation, and 10,000 test images; seed 42) under an identical 15-epoch Adam protocol, test Macro-F1 ranges from 0.8441 for Linear to 0.9163 for CNN. These results indicate that, for the configurations tested, classification performance depends strongly on how well an architecture's inductive bias preserves two-dimensional local image structure rather than on total parameter count alone.

Specifically, the two-stage CNN achieves the highest test accuracy (0.9165) and Macro-F1 (0.9163) across all 10 garment categories using 50,378 parameters—4.0 times fewer than the MLP and 5.4 times fewer than the Transformer. The row-recurrent GRU (0.9010 Macro-F1, 61,962 parameters) outperforms both the LSTM (0.8898) and the flattened MLP (0.8902, 203,530 parameters). Moreover, the inter-model Macro-F1 spread is 4.5 times wider within the four silhouette-sharing upper-body garments (Shirt, Pullover, Coat, and T-shirt/top) than across the remaining six categories, supporting the importance of localized 2D feature extraction for this task.

These findings are specific to the tested configurations and one random seed. The 15-epoch budget also leaves the MLP and Transformer still improving at the end of training, so the results do not establish a universal ranking of architecture families. Future work should repeat the comparison across multiple seeds, evaluate parameter-matched models, and test patch-based Transformer inputs with a longer, tuned training schedule. Within these limitations, the benchmark shows that preserving 2D locality is valuable for this small image-classification task, while compact recurrent models can remain competitive alternatives.

5 AI Usage Disclosure

In accordance with Sections 5.1 and 5.2 of the CO3133 Course Handbook, this section records the AI tools reported by the three members of Group CR7, their purposes, affected work, verification, and sources. The group also maintains a detailed disclosure record in `assignment1/AI_USAGE.md`.

5.1 Group Member Roles and AI Usage Overview

Table 6: Group contributions, reported AI use, and verification responsibilities.

Member	Assignment contributions	AI tools reported	Verification role
Trần Hoàng Vỹ Khang (2352502)	Data pipeline, training engine, metrics and plotting utilities, seed utilities, EDA notebook, integration and verification of six models, benchmark runs, comparison script, README, project page, and final report consolidation.	Claude Code CLI (Claude Sonnet 5 and Claude Opus 5); Google Antigravity Agent.	Primary verifier of code modules, run artifacts, figures, and report claims.
Đỗ Đăng Khoa (2352554)	Member-written Linear and MLP baselines for the M1 draft; prototype MLP–CNN branch exploration; verification of EDA, preprocessing, and Linear/MLP/CNN architecture and compute claims.	No generative AI used to author the M1 Linear/MLP code; code review and report refinement are recorded in Entries 1, 2, and 4.	Co-verified tensor shapes, parameter counts, FLOPs/MACs, and EDA statistics.
Dương Đăng Khoa (2352551)	Experimental verification of LSTM, GRU, and Transformer; auditing per-class F1 and confusion matrices; co-authoring and verifying training behavior, error analysis, recurrent comparison, and limitations.	No standalone AI code generation outside shared repository sessions; participated in reviewing Entries 3 and 4.	Co-verified sequence shapes, recurrent training histories, and confusion-matrix error counts.

5.2 Detailed Disclosure Entries

The entries below summarize the reported AI assistance and verification process. Each entry records the tool, user, stage, purpose, affected work, representative prompts or log location, treatment of AI output, responsible verification, and verification sources.

5.2.1 Entry 1: Review and Cleanup of the M1 Draft Training Pipeline

- **Tool and model:** Claude Code CLI, using Claude Sonnet 5 and Claude Opus 5.
- **Member and stage:** Trần Hoàng Vỹ Khang (2352502), reviewing work by Trần Hoàng Vỹ Khang and Đỗ Đăng Khoa during M1 Draft consolidation on 16 September 2026.

- **Purpose and category:** Code review, debugging, and repository hygiene; members manually applied the changes.
- **Affected work:** `src/utils/seed.py`, `src/data/dataset.py`, `src/utils/metrics.py`, `src/utils/plots.py`, `src/models/linear.py`, `src/models/mlp.py`, and `src/engine/trainer.py`; training/evaluation scripts; root ignore configuration. Report sections: 1.3–1.5, 2.1, 2.2.1–2.2.2, and 2.3.
- **Representative prompts and log:** Requests for code review/cleanup and for re-checking normalization and data splitting. The conversation log is in `assignment1/AI_USAGE.md` (Entry 1).
- **Editing and verification:** The member manually addressed the review findings, including a syntax/indentation issue in the seed utility, normalization and stratified splitting, a colorblind-safe plotting palette, and an anchored data ignore rule.
- **Responsible verification:** Trần Hoàng Vỹ Khang (2352502) and Đỗ Đăng Khoa (2352554).
- **Verification sources:** PyTorch and torchvision documentation, scikit-learn metric documentation, Course Handbook Sections 10–12, and empirical statistics in the EDA notebook.

5.2.2 Entry 2: Review of the Prototype MLP–CNN Branch

- **Tool and model:** Claude Code CLI, using Claude Sonnet 5 and Claude Opus 5.
- **Member and stage:** Trần Hoàng Vỹ Khang (2352502), reviewing the prototype branch authored by Đỗ Đăng Khoa (2352554), on 22 September 2026.
- **Purpose and category:** Architecture review and shape-test generation for the unmerged prototype branch.
- **Affected work:** The unmerged MLP–CNN prototype branch only; no results in the report come from this branch.
- **Representative prompt and log:** A request to review the MLP–CNN branch, recorded as Entry 2 in `assignment1/AI_USAGE.md`.
- **Editing and verification:** The output shape $(N, 10)$ and spatial-size rule (image size divided by two per pooling block) were checked for two, three, and four convolutional blocks. The group chose not to merge the prototype and instead implemented a new CNN for the final project.
- **Verification sources:** Course Handbook Sections 11.1 and 12.2; PyTorch Conv2d and MaxPool2d documentation.
- **Responsible verification:** Trần Hoàng Vỹ Khang (2352502) and Đỗ Đăng Khoa (2352554).

5.2.3 Entry 3: Review of M2 Models, Sequence Helper, and Timing Isolation

- **Tool and model:** Claude Code CLI, using Claude Opus 5.
- **Member and stage:** Trần Hoàng Vỹ Khang (2352502), with joint verification by Đỗ Đăng Khoa (2352554) and Dương Đăng Khoa (2352551), during 2–5 October 2026.
- **Purpose and category:** Architecture and code review, debugging determinism and latency measurement, and cross-checking benchmark summaries against run artifacts.
- **Representative prompts and log:** Requests to check M2 architectures against the course handbook and review model implementations and numerical consistency. The conversation log is in `assignment1/AI_USAGE.md` (Entry 3).
- **Editing and verification:** The group verified raw $(N, 10)$ logits without a softmax layer, row/column/patch sequence conversion, deterministic CNN behavior across repeated runs, and reported metrics against the saved configuration, history, and test-metric artifacts. The review itself did not directly modify the model code.
- **Responsible verification:** Trần Hoàng Vỹ Khang (2352502), Đỗ Đăng Khoa (2352554), and Dương Đăng Khoa (2352551).
- **Verification sources:** PyTorch documentation for the used neural-network layers, Course Handbook Sections 11.1–12.2, and committed JSON run artifacts.

5.2.4 Entry 4: Combined Figures and Scientific Audit of the Report

- **Tool:** Google Antigravity Agent.
- **Members and stage:** Trần Hoàng Vỹ Khang (2352502), Đỗ Đăng Khoa (2352554), and Dương Đăng Khoa (2352551), during final report verification in October 2026.
- **Purpose and category:** Review of the multi-model comparison script and combined learning-curve/confusion-matrix figures; scientific and numerical audit of report claims, tables, captions, error counts, and compute calculations.
- **Affected work:** The comparison script and combined figures; all report parts, including reconciling the CNN `Shirt`→`Pullover/Coat` count ($33 + 64 = 97$) and best validation scores against repository artifacts and project summaries.
- **Representative prompts and log:** Requests to reconcile the README, project page, report, and output JSON; and to audit error analysis, FLOPs/MACs, and conclusions against handbook requirements. The conversation log is recorded as Entry 4 in `assignment1/AI_USAGE.md`.
- **Editing and verification:** Group members applied the review suggestions and checked tables against history, test-metric, and configuration JSON files, including recurrent epoch-by-epoch comparisons and confusion-matrix cell sums. The figures were also visually reviewed by the group.
- **Responsible verification:** All three members of Group CR7.
- **Verification sources:** Committed output JSON, dataset and EDA artifacts, the disclosure record, and Course Handbook Sections 3.1–3.3, 5.2, and 10–13.